

From Widgets to World-Class: Proven Architectures and Next-Generation Practices in Professional Flutter Development

Performance Engineering: Achieving Fluidity and Responsiveness at Scale

As of early 2026, performance engineering in professional Flutter development is a proactive discipline focused on building fluid, jank-free user experiences from the ground up. It is no longer a post-development optimization but an integral part of the architecture and coding process. The primary objective is to consistently achieve and maintain 60 frames per second (FPS), which is perceived by users as perfectly smooth motion ³. This is achieved through a combination of disciplined widget tree management, strategic offloading of computational work, effective resource management, and the full utilization of modern rendering technologies like the Impeller engine.

A foundational practice for ensuring performance is the meticulous control of widget rebuilding. The Flutter framework rebuilds widgets in response to state changes, but not all rebuilds are necessary or efficient. The most fundamental technique is the judicious use of `const` constructors on widgets that do not change ³. By marking a widget as `const`, it becomes immutable, and the framework can reuse the same instance instead of recreating it during subsequent builds. This simple practice drastically reduces the workload on the Dart virtual machine and the rendering pipeline. For widgets that contain mutable state, developers employ more advanced strategies. `ListView.builder` and `GridView.builder` are non-negotiable patterns for displaying lists of data; they leverage virtualization to build only the items that are currently visible on the screen, preventing the creation of thousands of widget objects for a long list ³. To further optimize scrolling performance, providing a fixed `itemExtent` for list items with uniform height allows the framework to pre-calculate the layout without needing to measure each item individually, a computationally expensive operation ³. For complex widgets that repaint independently, `RepaintBoundary` is

used to isolate their paint operations from the rest of the widget tree, preventing cascading repaints that can cause significant jank [3](#) .

Heavy operations that block the main UI thread are a primary source of unresponsiveness. These include parsing large JSON payloads, performing complex mathematical calculations, or interacting with local databases synchronously [3](#) . In a production environment, any task that takes more than a few milliseconds must be moved off the main thread. The recommended solution is to execute this work within a separate isolate, a lightweight thread of execution in the Dart VM. The `compute()` function provides a simple API for running a one-off function in an isolate, while more complex scenarios may require managing a dedicated isolate pool to handle ongoing background tasks [3](#) . This ensures that the UI remains free to handle user input and render frames smoothly, even while intensive processing occurs in the background. Furthermore, memory management is critical for sustained performance, especially on devices with limited resources. Best practices involve using weak references for caches so that cached objects can be garbage collected if the system experiences low-memory pressure [26](#) . Developers should also listen for low-memory warnings sent by the operating system and respond by clearing caches or releasing non-essential resources [26](#) .

The maturation of the Impeller rendering engine has been a pivotal development for Flutter's performance profile. By 2026, Impeller has become the default rendering backend on most platforms, largely replacing the legacy Skia engine [25](#) . Impeller is built specifically for Flutter and renders directly to the GPU, bypassing some of the bottlenecks inherent in the older software-based rendering path. This architectural shift has yielded significant, measurable improvements. According to case study data, the adoption of Impeller has led to a 62% reduction in jank rates on low-end Android devices and a 37% improvement in first-screen load times for Flutter Web applications [31](#) . These gains make Flutter a viable option for performance-sensitive applications, from gaming to financial trading dashboards, where a buttery-smooth interface is not just a feature but a core requirement.

Finally, a data-driven approach to performance tuning is paramount. Before attempting to fix a performance issue, developers must first identify it with precision. The Flutter DevTools suite is the industry-standard toolset for this purpose [18](#) [33](#) . The Performance overlay in the Flutter Inspector provides a quick visual cue, highlighting frames that take longer than 16ms (the target for 60 FPS) [11](#) . For deeper analysis, the Performance tab within Flutter DevTools offers a detailed timeline view of every frame, allowing developers to drill down into the exact Dart and UI threads and pinpoint the source of the bottleneck, whether it's excessive widget rebuilding, long rasterization times, or other

issues [3](#) [11](#). The Widget Rebuild Stats feature helps identify which widgets are being rebuilt too frequently, guiding refactoring efforts towards more granular state management or better use of `const` [3](#). This emphasis on profiling before optimizing ensures that development effort is directed at actual, quantifiable problems rather than perceived issues, leading to more efficient and effective performance engineering.

State Management and Architectural Patterns for Scalability

In the professional Flutter ecosystem of 2026, the choice of state management and architecture is a critical decision that profoundly impacts the long-term health, scalability, and maintainability of an application. The era of a single "best" state management solution has passed; instead, there is a sophisticated consensus on selecting specialized tools tailored to project scale, regulatory requirements, and performance demands. Similarly, architectural patterns that enforce a strict separation of concerns have become the gold standard for enterprise-grade applications.

For state management, Riverpod 3.0 has emerged as the modern standard for the majority of new projects [7](#). Its popularity stems from several key advantages over earlier solutions. Firstly, it provides compile-time safety, eliminating the runtime errors that plagued earlier provider-based solutions where a missing provider could crash the app [7](#). Secondly, it significantly reduces boilerplate code, allowing developers to write cleaner and more concise state logic. Riverpod 3.0 also comes with powerful built-in features, such as an experimental offline persistence module that supports local hydration and stale-while-revalidate refresh strategies, and a Mutations API for defining asynchronous actions with clear lifecycle states (Idle, Pending, Success, Error), which simplifies complex UI interactions [7](#).

However, Riverpod is not the universal answer. In regulated industries like banking and healthcare, BLoC 9.0 remains the "enterprise standard" [7](#). Its event-driven nature creates a strict, auditable trail of every state change, which is often a mandatory requirement for compliance and rigorous debugging of complex systems [7](#). The latest version of BLoC has addressed common pain points that once made it cumbersome, such as `use-after-dispose` errors, through auto-mounted consumers like `BlocListener` that automatically check if the context is still active before executing callbacks [7](#). It also improves performance by allowing for granular filtering of state updates via the

BlocSelector, ensuring that only the widgets dependent on specific parts of the state are rebuilt 7 .

For applications where performance is the absolute top priority, such as real-time data dashboards or competitive games, Signals 6.0 is gaining significant traction 7 . It introduces a paradigm of fine-grained reactivity, enabling what is described as "surgical rendering." Unlike traditional state management where a state change triggers a cascade of widget rebuilds, Signals notify only the listeners that are directly dependent on the changed value, often bypassing the widget rebuild cycle entirely 7 . This makes it exceptionally performant for UIs driven by frequent, rapid-fire updates.

While these three represent the modern landscape, it is important to understand the roles of older solutions. Provider is still supported but is increasingly viewed as a legacy option for complex applications, best suited for learning purposes or maintaining existing codebases that do not require advanced features 7 . Conversely, GetX is actively discouraged for new professional projects due to significant architectural flaws and maintenance risks, including its reliance on global singletons, which complicates testing and debugging, and a maintenance crisis stemming from its dependence on a single maintainer 7 .

Feature	Riverpod 3.0	BLoC 9.0	Signals 6.0
Primary Use Case	Most new applications 7	Regulated industries (finance, healthcare) 7	High-performance, reactive UIs (dashboards, games) 7
Key Strength	Compile-time safety, minimal boilerplate, rich features 7	Strict audit trails, mature pattern 7	Fine-grained reactivity ("surgical rendering") 7
Boilerplate	Minimal 7	Moderate 7	Low (with SignalsMixin) 7
Error Handling	Structured via Result pattern or try/catch 7	Event-driven, explicit state propagation 7	Unified loading/error/data via FutureSignal 7

Architecturally, Clean Architecture is the preferred pattern for large-scale, enterprise-grade Flutter applications 5 . Its core principle is the Dependency Inversion Principle: high-level modules (business logic) should not depend on low-level modules (data sources); both should depend on abstractions (interfaces) 6 . This decoupling provides maximum flexibility and testability. A typical implementation involves a feature-based modularization structure with a top-level features/ directory 6 . Each feature is its own self-contained module with data, domain, and presentation layers. The domain layer contains business logic, entities, and use cases, and it has no knowledge of the data layer. The data layer implements repository contracts defined in the domain layer to fetch data from external sources like APIs or local databases. The

presentation layer handles the UI and interacts with the domain layer's use cases via controllers or ViewModels [6](#) . This strict separation of concerns is critical for team scalability, as different teams can work on the UI and business logic in parallel without stepping on each other's toes [7](#) .

Model-View-ViewModel (MVVM) is a popular and effective design pattern for implementing Clean Architecture in Flutter [5](#) . In this pattern, the View is the UI (typically a `StatelessWidget` or `StatefulWidget`), the Model represents the data, and the ViewModel acts as the intermediary. The ViewModel exposes streams or values that the View consumes, often using the `provider` package for dependency injection and state propagation [5](#) . It orchestrates calls to the domain layer's use cases and formats the returned data into a UI-friendly format. This pattern promotes a clean separation of concerns, making the codebase highly maintainable, scalable, and testable [5](#) .

Crafting Polished UI/UX with Modern Design Principles

Achieving exceptional UI/UX in professional Flutter applications as of 2026 extends beyond mere aesthetics; it involves a deep understanding and practical application of modern design languages, adaptive layout techniques, performant animation, and rigorous accessibility standards. These elements combine to create visually polished, intuitive, and inclusive user experiences that are crucial for user engagement and retention.

The foundation of modern UI design in Flutter is the implementation of Material Design 3 (MD3), also known as Material You [17](#) . MD3 moves away from rigid design systems toward a more personalized and dynamic approach. Key principles include the use of dynamic color, which generates a harmonious color palette based on the user's wallpaper, and robust typography and shape systems that provide consistency across the app. Developers leverage packages like `flex_color_scheme` to easily define beautiful, material-design-based themes [17](#) . On iOS, developers adhere strictly to the Cupertino design guidelines, using widgets from the `cupertino` library to ensure a native feel for Apple platform users [1](#) . The goal is to create a unified brand identity while respecting the platform-specific conventions of Android and iOS.

Adaptive and responsive layouts are a non-negotiable requirement for any app targeting a wide range of devices, from small phones to tablets and desktops. Flutter provides a powerful toolkit for achieving this responsiveness. Developers commonly use `LayoutBuilder` and `MediaQuery` to make UI decisions based on the available screen space [24](#). `LayoutBuilder` provides constraints about the parent widget's size, allowing for conditional widget trees—for example, switching from a vertical column to a horizontal row on wider screens. `MediaQuery` offers direct access to device information like screen width and orientation [24](#). For more complex responsive needs, third-party packages like `Sizer` have gained popularity. `Sizer` simplifies responsive design by providing a context-aware builder that automatically adapts the UI to the screen size, making it easier to scale fonts, padding, and margins proportionally without writing extensive conditional logic [23](#). The overarching goal is to ensure the application is usable and visually appealing regardless of the device form factor, a challenge that is becoming increasingly important with the rise of foldable devices and larger screen sizes [21](#) [29](#).

Smooth animations and micro-interactions are vital for creating a sense of polish and providing feedback to the user. While Flutter's core animation classes like `AnimatedBuilder` offer fine-grained control for custom animations, many developers turn to specialized packages for efficiency and ease of use. Lottie and Rive are two of the most popular choices for integrating high-quality, vector-based animations created by designers [3](#). These packages allow developers to easily embed complex animations into the UI, enhancing the overall experience without writing extensive animation controller code. For bespoke, highly optimized animations that require maximum performance, custom painters remain a powerful option, though they demand a deeper understanding of Flutter's rendering pipeline [3](#).

Accessibility (a11y) is no longer an afterthought but a core component of the development process. Ensuring an application is usable by people with disabilities is both an ethical imperative and, in some regions, a legal requirement. Key best practices include assigning meaningful semantic labels to widgets to help screen readers describe the UI to visually impaired users. Developers must also pay close attention to color contrast ratios, ensuring sufficient distinction between text and background colors to aid users with low-vision conditions [1](#). Another critical aspect is supporting dynamic text scaling, allowing users to adjust the system font size, which the application must respect gracefully. Failure to do so can result in clipped text or broken layouts, rendering the app unusable for certain users. Comprehensive accessibility testing is an essential part of the quality assurance workflow for any professional Flutter application.

Platform Integration and Cross-Platform Nuances

While Flutter's primary value proposition is a single codebase for multiple platforms, professional development requires a nuanced approach to platform integration. Seamless interaction with native capabilities, adherence to platform-specific conventions, and adaptation to OS-level changes are critical for delivering a truly native-feeling and robust application. The goal is to abstract away platform differences where possible while providing efficient mechanisms for bridging gaps where they are unavoidable.

When a Flutter-native solution is insufficient, developers use platform views to embed complex native UI components directly into the Flutter widget tree [3](#). This is particularly useful for integrating third-party libraries that provide native-only functionality, such as advanced mapping services or high-performance video players [3](#). On Android, this is accomplished with the `AndroidView` widget, and on iOS, with `UIKitView`. The mechanism involves creating a `PlatformViewFactory` to generate the native view and manage its lifecycle [3](#). However, this power comes with significant trade-offs. Communication between the Flutter framework and the native view is asynchronous, which means data synchronization must be handled carefully to avoid race conditions and stale data displays [3](#). Furthermore, debugging can be challenging as it often requires switching contexts between the Dart and native (Java/Kotlin or Swift/Objective-C) codebases [3](#).

For bridging functional gaps between Dart and native code—such as accessing a unique sensor, using a proprietary SDK, or performing a low-level hardware operation—platform channels are the standard mechanism. `MethodChannel` is used to send synchronous messages to the native side, while `EventChannel` is used for streaming data back to the Dart side [3](#). A well-designed platform channel interface is crucial for maintainability. It should expose a clean, well-documented Dart API that abstracts away the complexities of the native implementation, making the native code feel like a first-class citizen within the Flutter application. For tasks requiring zero-latency native responses, such as listening for specific button press sequences, the core logic should be implemented in native code and then exposed to Flutter via `MethodChannel`, rather than relying on potentially slower Dart packages [3](#).

Professional developers must also stay vigilant about changes in the underlying operating systems. A prime example is the upcoming Android 16, which will prohibit apps from locking themselves to portrait mode, forcing developers to design flexible, orientation-aware interfaces [15](#). Similarly, the evolution of Flutter Desktop requires developers to consider multi-window support, improved keyboard and mouse handling, and proper

focus management, features enhanced in Flutter 3.32 [29](#). Adapting to these changes is not optional; it is essential for ensuring the application remains compliant and provides a modern user experience on all target platforms. This ongoing need for adaptation underscores the importance of cross-platform development skills that go beyond the Flutter framework itself.

The Evolving Developer Experience: Tooling and Workflow

The developer experience is a cornerstone of Flutter's appeal, and in 2026, it has matured into a comprehensive ecosystem of tools and workflows designed to accelerate development, facilitate debugging, and ensure code quality. Professional developers rely on a suite of official and community-driven tools to navigate the complexities of building and maintaining large-scale applications efficiently.

The Flutter DevTools suite is an indispensable part of the daily workflow for any serious Flutter developer [18](#) [33](#). It is a collection of web-based UIs for inspecting, debugging, and profiling Flutter applications. The **Performance** tab is critical for analyzing application responsiveness, offering a detailed timeline of frames to identify jank and long-running operations [11](#). The **Memory** tab allows developers to track object allocations over time and inspect heap snapshots, which is invaluable for diagnosing memory leaks—a common problem in long-running applications [33](#). The **Layout Inspector** provides a visual representation of the widget tree, enabling developers to debug layout issues by inspecting constraints, sizes, and alignments of widgets in the UI hierarchy [18](#). For network-heavy applications, the `flutter_network_inspector` package provides a dedicated tool to monitor HTTP requests made using Dart's native `http` package, helping to debug connectivity issues and optimize data fetching [12](#).

The Hot Reload feature remains a central pillar of the Flutter development experience, praised for its ability to inject code changes into a running Dart VM in under two seconds [19](#). This rapid feedback loop dramatically accelerates the iterative development process. However, as applications grow in complexity, developers have refined their workflows around it. They structure their code into small, logical, and self-contained units to maximize the effectiveness of Hot Reload. When changes are too extensive for a hot reload—for example, modifying class hierarchies or adding new dependencies—a hot restart is performed. This re-initializes the app state but is still much faster than a full

rebuild and redeploy from an IDE. Understanding when to use each is a key skill for maintaining a productive development pace.

Maintaining a high standard of code quality across a large team is managed through rigorous linting and static analysis. Packages like `lint` or `very_good_analysis` provide pre-configured sets of rules that enforce a consistent code style, catch potential bugs before they reach production, and ensure adherence to best practices [33](#). These configurations are integrated into the development workflow through IDE plugins and CI/CD pipelines, ensuring that every pull request meets the project's quality standards. This automated enforcement of quality prevents the accumulation of technical debt and makes the codebase more readable and maintainable over time.

Emerging Frontiers: AI, Dynamic Theming, and the Future of Flutter

The period of 2025–2026 marks a significant inflection point for the Flutter ecosystem, characterized by the integration of artificial intelligence, continued advancements in rendering technology, and a deeper embrace of dynamic and adaptive user experiences. These emerging trends are moving Flutter beyond a simple cross-platform toolkit into a more intelligent, performant, and future-ready development platform.

One of the most impactful developments is the emergence of AI-assisted development. At Google I/O 2025, Google introduced the official GenUI SDK, signaling a major step forward in this area [34](#). This SDK moves beyond simple prompt-to-code generators by integrating directly into the Flutter toolchain. It allows developers to describe desired UI components in natural language, and the framework generates the corresponding Dart code. This has the potential to dramatically accelerate prototyping, automate repetitive UI tasks, and lower the barrier to entry for less experienced developers, fundamentally changing the way UIs are conceptualized and built.

Advancements in rendering continue to push the boundaries of performance. As of 2026, the Impeller rendering engine has become the mainstream standard, having largely solved the performance inconsistencies of the legacy Skia engine [25](#). Its direct-to-metal rendering approach has resulted in tangible, quantifiable benefits, such as a reported 62% decrease in jank on low-end mobile devices [31](#). This solidifies Flutter's position for demanding applications and allows developers to build smoother, more responsive

interfaces out-of-the-box. Concurrently, support for new device form factors like foldables and tablets is maturing. Flutter Desktop 3.32 introduced enhanced multi-window support, requiring developers to think more deeply about desktop UX patterns involving keyboard, mouse, and focus ²⁹.

The principles of Material You have matured into a set of powerful, practical tools for creating dynamic and personalized user experiences. Developers are now leveraging dynamic color, real-time personalization, and declarative theming more extensively. Packages like `flex_color_scheme` provide sophisticated tools for generating beautiful, adaptable Material Design themes that can respond to user preferences, content, and even time of day ¹⁷. This trend is further fueled by growing support for adaptive layouts and dynamic theming, which are essential for delivering a cohesive and engaging experience across the diverse array of modern devices ²⁹. Beyond these proven patterns, developers are also exploring experimental frontiers, such as using AI to create adaptive interfaces that learn from user behavior and employing custom painters for hyper-optimized, bespoke animations that surpass what general-purpose libraries can offer ³. This continuous exploration signals a vibrant and forward-looking community that is actively shaping the future of mobile development with Flutter.

Reference

1. Flutter in Production: Real-World Use Cases https://www.linkedin.com/posts/suhana-kumari-94a538236_flutter-flutterdev-mobiledevelopment-activity-7413153047243063296-Y8XM
2. Modern Frontend Stacks in 2025: Why the JAMstack Is (Still ... <https://www.linkedin.com/pulse/modern-frontend-stacks-2025-why-jamstack-still-worth-ayan-sarkar-f62zf>
3. Flutter Platform Views for Native Code Customization https://www.linkedin.com/posts/arab-kumar-b5853b191_flutter-dart-flutternews-activity-7413089290118950912-jI5P
4. Choosing the Right Architecture: MVC, MVP, MVVM, MVI, ... https://www.linkedin.com/posts/ashsau_mvc-vs-mvp-vs-mvvm-vs-mvi-vs-viper-which-activity-7363762365009448960-W0BN

5. Flutter MVVM Architecture Explained: A Complete Guide ... <https://www.linkedin.com/pulse/flutter-mvvm-architecture-explained-complete-guide-2025-chaturvedi-yzhqf>
6.  Flutter Clean Architecture - Fast Feature-First Setup Guide <https://dev.to/shanshaji/flutter-clean-architecture-fast-feature-first-setup-guide-342l>
7. Best Flutter State Management Libraries 2026 <https://foresightmobile.com/blog/best-flutter-state-management>
8. Flutter vs React Native: Best Cross-Platform Framework in ... <https://www.shaligraminfotech.com/flutter-vs-react-native-comparison/>
9. Hermans K. Mastering Flutter. A Comprehensive Guide To ... <https://www.scribd.com/document/911255746/Hermans-K-Mastering-Flutter-a-Comprehensive-Guide-to-Learn-Flutter-Devel-2023>
10. From Models to Momentum: How Alibaba Turned Artificial ... https://www.alibabacloud.com/blog/from-models-to-momentum-how-alibaba-turned-artificial-intelligence-into-a-real-world-utility-in-2025_602797
11. Rahul Naik's Post https://www.linkedin.com/posts/rahula-naika_flutter-flutterdev-flutterwidgets-activity-7392085571029848064-DmNp
12. flutter_network_inspector | Flutter package https://pub.dev/packages/flutter_network_inspector
13. Centurion Mechanical 2024 | PDF | Probability Distribution <https://www.scribd.com/document/885307700/Centurion-Mechanical-2024>
14. 1 - Amazon S3 <https://s3.eu-central-1.amazonaws.com/deepset.ai-farm-models/0.4.1/glove-english-uncased-6B/vocab.txt>
15. Android 16: Flutter Apps Must Adapt to Portrait & Landscape https://www.linkedin.com/posts/muhammadfaisaldev_flutter-android-android16-activity-7365474903279493120-RnZn
16. How to Test Flutter App Across Screens & Platforms https://www.linkedin.com/posts/hassanzaheer21_fluttertips-devlife-mobileappdev-activity-7371807225713029120-uLhn
17. flex_color_scheme changelog | Flutter package https://pub.dev/packages/flex_color_scheme/changelog
18. Flutter DevTools 全景介绍 <https://zhuanlan.zhihu.com/p/1972755038691673175>
19. Flutter热重载（Hot Reload）解析 - 开源鸿蒙跨平台开发者社区 <https://openharmonyxplatform.csdn.net/69412da496fa167eecea56c.html>
20. glove.6B.100d.txt-vocab.txt <https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt>

21. Mastering Responsive UIs in Flutter: The Full Guide <https://dev.to/daridigregorio/mastering-responsive-uis-in-flutter-the-full-guide-3g6i>
22. How to Make a Flutter App Responsive for Different Mobile ... <https://stackoverflow.com/questions/79539122/how-to-make-a-flutter-app-responsive-for-different-mobile-screen-sizes>
23. sizer 3.1.2 | Flutter package <https://pub.dev/packages/sizer/versions/3.1.2>
24. How to use LayoutBuilder and MediaQuery in Flutter https://www.linkedin.com/posts/karan-500700250_responsive-design-in-flutter-why-layoutbuilder-activity-7346609266960605184-NH5N
25. How Impeller Is Transforming Flutter UI Rendering in 2026 <https://dev.to/eira-wexford/how-impeller-is-transforming-flutter-ui-rendering-in-2026-3dpd>
26. How to Build Scalable and Performant Flutter Applications <https://www.freecodecamp.org/news/how-to-build-scalable-and-performant-flutter-applications-a-handbook-for-devs/>
27. FORD Acronyms Booklet | PDF | Annual Percentage Rate <https://www.scribd.com/doc/94341190/FORD-Acronyms-Booklet>
28. Continue | PDF | Gear | Axle <https://www.scribd.com/document/716893476/83e9efa9ff6a4-4>
29.  Flutter Desktop: Beyond Mobile — A Complete Guide for ... <https://www.linkedin.com/pulse/flutter-desktop-beyond-mobile-complete-guide-fabr%C3%ADcio-ferreira-ud1rf>
30. Lynx: A New Cross-Platform Framework by ByteDance https://www.linkedin.com/posts/irajputsourav_reactlynx-bytedance-crossplatform-activity-7304560747806629888-dJx6
31. Flutter 2025 深度解析：从Impeller 渲染革命到企业级架构 ... https://blog.csdn.net/2501_94436368/article/details/155953196
32. Navy Removal Scout 800 Pink Pill Assassin Expo Van ... <https://www.scribd.com/document/531005187/70048773907-navy-removal-scout-800-pink-pill-assasin-expo-van-travel-bothell-punishment-shred-norelco-district-ditch-required-anyhow>
33. Flutter - DevTools <https://www.geeksforgeeks.org/flutter/flutter-devtools/>
34. December 2025: Flutter GenUI SDK, Build Runner ... <https://codewithandrea.com/newsletter/december-2025/>